R: limpieza, manipulación y exploración de datos

Cheatsheet de `forcats`

Jonatán Perren

24 de marzo de 2026

Introducción a forcats

En R, las variables categóricas fijas se manejan como factores (`factor`). A diferencia de un texto libre (`character`), un factor tiene un conjunto predefinido y cerrado de valores posibles llamados niveles (`levels`).

`forcats` es parte del entorno «tidy» de R y resuelve todos los problemas que solían ser un dolor de cabeza al ordenar o agrupar variables categóricas, especialmente a la hora de graficar con `ggplot2`.

```
library(tidyverse) # Viene incluido y se carga por defecto
# o de forma explícita:
library(forcats)
```

Inspección y creación base

Cuando convertimos un texto a factor, R ordenará sus niveles alfabéticamente por defecto. Para los humanos, los datos parecen texto, pero R los ve internamente como números enteros indexados (1, 2, 3...).

```r
f <- factor(c("Manzana", "Pera", "Manzana", "Banana"))

# Veamos qué niveles descubrió R automáticamente:
levels(f)

# Inspección integral tipo tabla de frecuencias
fct_count(f, sort = TRUE) # (sort = TRUE ordena de mayor a menor frecuencia)
```

```
> levels(f)
[1] "Banana"  "Manzana" "Pera"
> fct_count(f, sort = TRUE)
# A tibble: 3 x 2
  f           n
  <fct>   <int>
1 Manzana     2
2 Banana      1
3 Pera        1
```

Cambiar el orden

El objetivo de cambiar el orden de los niveles **no** es cambiar los datos de tus filas, sino alterar el orden en que las categorías y barras aparecen listadas en un gráfico o en una tabla.

```r
# Movimiento manual estricto: forzar "Pera" y "Banana" al frente de la fila
f_manual <- fct_relevel(f, "Pera", "Banana")

# Ordenar automáticamente de mayor a menor frecuencia de aparición
f_frecuencia <- fct_infreq(f)

# Dar vuelta cualquier factor (invertir sus niveles de fin a principio)
f_invertido <- fct_rev(f_frecuencia)
```

```
> levels(f_manual)
[1] "Pera"    "Banana"  "Manzana"
> levels(f_frecuencia)
[1] "Manzana" "Banana"  "Pera"
> levels(f_invertido)
[1] "Pera"    "Banana"  "Manzana"
```

fct_reorder

La función más útil de todo **forcats**. Permite ordenar un factor (f) según los valores numéricos de **otra** variable matemática (x). Es lo que necesitás para crear un gráfico de barras pero que tus barras queden crecientes según el promedio de ingresos.

```r
grupo   <- factor(c("A", "B", "C", "A", "B", "C"))
ingreso <- c(100, 200, 50, 110, 210, 60)

# Por defecto, grupo se ordenó alfabéticamente: A, B, C.
# Forcemos el orden de "grupo" según la MEDIANA (default) del "ingreso":
grupo_ordenado <- fct_reorder(grupo, ingreso, .fun = median)
```

```
> levels(grupo)
[1] "A" "B" "C"
> levels(grupo_ordenado)
[1] "C" "A" "B"
```

Cambiar los valores de las etiquetas (Recodificación)

Si usás `fct_recode()` podés reescribir los nombres de las categorías manualmente, ideal para corregir faltas de ortografía o códigos crípticos. Se pasa primero el vector crudo, y luego las traducciones asumiendo la forma `"Nuevo" = "Viejo"`.

```r
f_mal <- factor(c("arg", "br", "br", "ch", "arg"))

# Recodificar nombre a nombre de forma ultra explícita
f_bien <- fct_recode(f_mal,
  "Argentina" = "arg",
  "Brasil"    = "br",
  "Chile"     = "ch"
)
```

```
> f_bien
[1] Argentina Brasil    Brasil    Chile     Argentina
Levels: Argentina Brasil Chile
```

Agrupar categorías conocidas (`fct_collapse`)

Si querés colapsar muchos niveles distintos en uno solo que funcione de «padre» lógico (ejemplo: agrupar pases provinciales en regiones macro), usás `fct_collapse()`.

```r
provs <- factor(c("Neuquén", "Santa Fe", "Chubut", "Córdoba", "Salta"))

# Recibe vectores para definir qué viejos entran en The Nuevo
regiones <- fct_collapse(provs,
  "Patagonia" = c("Neuquén", "Chubut"),
  "Centro"    = c("Santa Fe", "Córdoba"),
  "Norte"     = "Salta"
)
```

```
> regiones
[1] Patagonia Centro    Patagonia Centro    Norte
Levels: Centro Norte Patagonia
```

Agrupar la minoría

A veces tu factor tiene docenas de categorías y, a los fines del gráfico comercial, solo te importan las 3 principales. El resto querés que vayan sumadas a un tacho común bajo el nombre genérico «Otros».

```r
partidos <- factor(c(rep("Azul", 50), rep("Rojo", 40), rep("Verde", 5), "Gris", "Rosa"))

# Quedarse estrictamente con los 2 partidos top y el excedente arrojarlo a "Otro"
partidos_top2 <- fct_lump_n(partidos, n = 2, other_level = "Otro partido")

# O quedarse solo con las categorías cuya proporción supere el 10% del total
partidos_prop <- fct_lump_prop(partidos, prop = 0.10, other_level = "Minorías")
```

```
> fct_count(partidos_top2)
# A tibble: 3 x 2
  f                n
  <fct>        <int>
1 Azul            50
2 Rojo            40
3 Otro partido     7
```

Añadir y quitar niveles

Si partimos de un dataset global gigante y usamos `filter()` en la clase 5 para quedarnos solo con la gente de la provincia de Santa Fe, vas a notar que la columna de provincia ¡sigue guardando y mostrando todas las demás provincias de Argentina en tu tabla/gráfico (con barras vacías)!

```r
color <- factor(c("Rojo", "Azul", "Blanco"))

# Supongamos que perdemos la fila de datos "Blanco" por culpa de un filtro ciego:
color_filtrado <- color[1:2]

# Para purgar permanentemente los niveles vacíos zombis y curar el diccionario:
color_purgado <- fct_drop(color_filtrado)
```

```
> levels(color_filtrado)
[1] "Azul"   "Blanco" "Rojo"
> levels(color_purgado)
[1] "Azul" "Rojo"
```

Añadir niveles ciegos (`fct_expand`)

El caso radicalmente contrario también existe: a veces necesitás garantizar contractualmente que un nivel aparezca impreso en tu gráfico y en su leyenda, aunque en la realidad tu base de datos recopilada hoy esté transitoriamente vacía de esa categoría.

```r
turnos_hoy <- factor(c("Mañana", "Tarde"))

# Si la clínica tiene también turno "Noche" pero justo hoy nadie sacó,
# R no va a dibujar la barra de la noche. Vamos a crear el fantasma forzosamente.
turnos_totales <- fct_expand(turnos_hoy, "Noche")
```

```
> levels(turnos_totales)
[1] "Mañana" "Tarde"  "Noche"
```

Las funciones puras de `forcats` operan sobre **vectores**. Si querés aplicar estos cambios a escala industrial sobre una columna de tu tabla de datos real, tenés que usarlas encadenadas adentro de un verbo `mutate()`, reemplazando tu antigua columna con el nuevo `fct` arreglado.